

Safe Reinforcement Learning Using Robust Control Barrier Functions

Yousef Emam^{ID}, Gennaro Notomista^{ID}, *Member, IEEE*, Paul Glotfelter^{ID}, Zsolt Kira^{ID}, *Senior Member, IEEE*, and Magnus Egerstedt^{ID}, *Fellow, IEEE*

Abstract—Reinforcement Learning (RL) has been shown to be effective in many scenarios. However, it typically requires the exploration of a sufficiently large number of state-action pairs, some of which may be unsafe. Consequently, its application to safety-critical systems remains a challenge. An increasingly common approach to address safety involves the addition of a safety layer that projects the RL actions onto a safe set of actions. In turn, a difficulty for such frameworks is how to effectively couple RL with the safety layer to improve the learning performance. In this letter, we frame safety as a differentiable robust-control-barrier-function layer in a model-based RL framework. Moreover, we also propose an approach to modularly learn the underlying reward-driven task, independent of safety constraints. We demonstrate that this approach both ensures safety and effectively guides exploration during training in a range of experiments, including zero-shot transfer when the reward is learned in a constraint-agnostic fashion.

Index Terms—Robust/Adaptive control, reinforcement learning, robot safety.

I. INTRODUCTION

REINFORCEMENT Learning (RL) has been used successfully in numerous robotics applications [1], [2]. Typically, an RL agent must explore a sufficiently large number of states during training to learn a meaningful policy. However, this exploration may involve visiting unsafe states, making RL unsuitable for *safety-critical* systems with a high cost of failure [3], [4]. *Safe reinforcement learning* aims to tackle this challenge by combining control-theoretic techniques with RL as highlighted in [5]. A large variety of safe-RL approaches have been proposed in the literature. These approaches can be categorized as obtaining a safe policy *at the end of training* (e.g., [6], [7], [8], [9]) or, including the work in this letter, focusing on safe exploration *during training* (e.g., [10], [11], [12]). For example, in [13], the

authors propose a Control Barrier Function (CBF) and Control Lyapunov Function (CLF) based parametrized controller, where the parameters correspond to the outputs of a RL policy trained to estimate the uncertainty in the CLF and CBF constraints for a disturbed dynamical system. Although this approach yields a safe policy at the end of training, it does not guarantee safety throughout training.

Safe training for RL inherently consists of two components: a constraint-based component, which ensures the agent's safety, and a performance metric, typically captured by a reward or cost function, which the agent aims to optimize. In the work from [12], [14], the authors propose a Lyapunov-based approach for solving Constrained MDPs (CMDPs) [15] that guarantees the safety of the agent during training. Another similar approach is the work in [11] where the authors provide a model-based RL framework for safely expanding a region of attraction for the agent. This region of attraction is a set of states for which a stabilizing policy is known. However, the main drawback of these approaches is that complex environments induce intractability. Specifically, the method in [11] relies on a discretization of the state space via level sets of the Lyapunov function, an approach which cannot scale to high-dimensional states. Along similar lines, the approach in [12] relies on an optimization at each step with as many constraints as there are states, each of which involves an integral over the entire action space.

Another category of approaches leverages a model-based safety framework which serves to prevent the exploration of unsafe states (e.g., [16], [17]) by projecting the action taken by the RL agent onto a safe-set of actions. In turn, a central challenge to these approaches is how to leverage the knowledge of the dynamics and the model-based safety component to effectively guide exploration during training. Towards this end, one such approach is that in [10], where the authors assume knowledge of a CBF and a prior on the dynamics model. During training, a learned Gaussian Process (GP) represents the uncertainty in the dynamics and is accounted for in a Discrete-Time CBF (DT-CBF) based safety layer. Moreover, this work introduces a novel supervised-learning component to the policy, referred to by the authors as the compensator, which is a neural net that is used to learn the output of the CBF.

However, the approach in [10] suffers from three drawbacks. First, as highlighted in [18], the DT-CBF formulation used is restrictive, as it is amenable only to affine CBFs (i.e., CBFs of the form $h(x) = p^T x + q$ where x is the state). For example, in the context of collision avoidance, affine CBFs can only encode polytopic obstacles. Moreover, the supervised learning of the CBF layer fundamentally produces an approximation that, in the worst case, may negatively affect RL agent's learning if inaccurate. Lastly, the modifications to the actions made by the

Manuscript received 22 June 2022; accepted 10 October 2022. Date of publication 25 October 2022; date of current version 14 February 2025. This letter was recommended for publication by Associate Editor T. Osa and Editor J. Kober upon evaluation of the reviewers' comments. This work was supported by Army Research Lab through ARL DCIST CRA under Grant W911NF-17-2-0181. (Corresponding author: Yousef Emam.)

Yousef Emam and Zsolt Kira are with the Institute for Robotics and Intelligent Machines, Georgia Institute of Technology, Atlanta, GA 30332 USA (e-mail: emamy@gatech.edu; zsolt.kira@gtri.gatech.edu).

Gennaro Notomista is with the Department of Electrical and Computer Engineering, University of Waterloo, Waterloo, ON N2L3G1, Canada (e-mail: gennaro.notomista@uwaterloo.ca).

Paul Glotfelter is with the Outrider, Golden, CO 80403 USA (e-mail: pglotfel@gmail.com).

Magnus Egerstedt is with the Samueli School of Engineering, University of California, Irvine, CA 92697 USA (e-mail: magnus@uci.edu).

Digital Object Identifier 10.1109/LRA.2022.3216996

CBF layer and supervised learning components are not explicitly accounted for in the loss functions of the RL algorithm. This choice is equivalent to the effects of the CBF layer being treated as part of the unknown transition function with respect to the RL algorithm and can thus impede learning in cases where the agent behaves aggressively to avoid safety violations.

To address these challenges, by leveraging the work in [19], [20] where the authors propose a framework for backpropagation through convex optimization programs, we introduce a novel differentiable Robust CBF (RCBF) [21] based safety layer that is compatible with standard policy-gradient RL algorithms. Specifically, the proposed differentiable RCBF layer enables the mini-batch gradient updates of the RL policy to be taken with respect to the final safe action. As such, the proposed framework explicitly accounts for the behavior of the safety layer, which improves the training performance as demonstrated empirically in Section IV. Moreover, the formulation of the RCBF based-safety layer results in a Quadratic Program (QP) for any control-affine system even if the barrier functions are non-affine, and can encode a wide class of disturbances making the proposed framework computationally applicable to a large variety of systems.

Moreover, it may be the case that the CBF constraints differ during training and testing. Such cases may include a drone that must remain within a given distance from a safety operator during training in applications such as construction [22] or entertainment [23]. Training and testing constraints also differ when treating RL as a persistent task that involves charging constraints [24] or when using multiple agents to speed up training (e.g., [25]), in which case inter-agents collisions must be avoided. In such scenarios, the learned value function at the end of the training needs to be independent of a subset of the CBF constraints imposed during training. Towards this end, we propose a novel constraint-agnostic approach for learning the reward-driven task, which is empirically validated in Section IV. Specifically, we test the zero-transfer ability of various approaches in multiple environments, demonstrating that the proposed approach significantly improves performance.

II. BACKGROUND MATERIAL

In this section, we introduce CBFs and their robust counterpart from [21] which can be embedded as a wrapper around any nominal controller to ensure the safety of a disturbed dynamical system. Then, we discuss the underlying RL algorithm, namely Soft Actor Critic (SAC) [26]. The motivation behind using SAC is its sample-efficiency; however, we note that the methods proposed in this letter are compatible with any off-the-shelf policy-gradient algorithm.

We consider the following disturbed control-affine system

$$\dot{x}(t) = f(x(t)) + g(x(t))u(x(t)) + d(x(t)), \quad (1)$$

where $f: \mathbb{R}^n \rightarrow \mathbb{R}^n$ denotes the drift dynamics, $g: \mathbb{R}^n \rightarrow \mathbb{R}^{n \times m}$ is the input dynamics, $d: \mathbb{R}^n \rightarrow \mathbb{R}^n$ is an unknown, continuous and deterministic disturbance and $u: \mathbb{R}^n \rightarrow \mathbb{R}^m$ is the input signal. We assume that f , g , d and u are continuous. We note that the work in this letter can be straightforwardly extended to the case where d is stochastic since GPs are used to learn the disturbance.

A. Robust Control Barrier Functions

Control barrier functions [27], [28], [29] are formulated with respect to control-affine systems

$$\dot{x}(t) = f(x(t)) + g(x(t))u(x(t)), x(0) = x_0. \quad (2)$$

A set $\mathcal{C}$ is called forward invariant with respect to (2) if given a (potentially non-unique) solution to (2) $x: [0, t_1] \rightarrow \mathbb{R}^n$, $x_0 \in \mathcal{C} \Rightarrow x(t) \in \mathcal{C}, \forall t \in [0, t_1]$. Note that (2) is the unperturbed version of (1).

Since we are interested in guaranteeing the safety of the disturbed dynamical system (1), we leverage RCBFs [21], [30], which generalize the notion of CBFs to systems obeying the following differential inclusion

$$\dot{x}(t) \in f(x(t)) + g(x(t))u(x(t)) + D(x(t)), x(0) = x_0, \quad (3)$$

where f, g, u are as in (1) and $D: \mathbb{R}^n \rightarrow 2^{\mathbb{R}^n}$ (the disturbance) is an upper semi-continuous set-valued map that takes nonempty, convex, and compact values. Note that $2^{\mathbb{R}^n}$ refers to the power set of $\mathbb{R}^n$ and that the assumptions made on the disturbance are conditions to guarantee the existence of solutions [31].

Moreover, it was shown that for a specific form of D , we can recover a similar formulation of regular CBFs with almost no additional computational cost as stated in the following theorem. An important aspect of this theorem is that forward invariance is guaranteed for all trajectories of (3).

Theorem 1: [21] Let $h: \mathbb{R}^n \rightarrow \mathbb{R}$ be a continuously differentiable function. Let $\psi_i: \mathbb{R}^n \rightarrow \mathbb{R}^n$, $i \in \{1, \dots, p\}$ be a set of $p > 0$ continuous functions, and define the disturbance $D: \mathbb{R}^n \rightarrow 2^{\mathbb{R}^n}$ as

$$D(x') = \text{co } \Psi(x') = \text{co}\{\psi_1(x') \dots \psi_p(x')\}, \forall x' \in \mathbb{R}^n,$$

where co denotes the convex hull. If there exists a continuous function $u: \mathbb{R}^n \rightarrow \mathbb{R}^m$ and a locally Lipschitz extended class- $\mathcal{K}$ function $\gamma: \mathbb{R} \rightarrow \mathbb{R}$ such that

$$\begin{aligned} L_f h(x') + L_g h(x')u(x') &\geq -\gamma(h(x')) \\ &\quad - \min \nabla h(x')^\top \Psi(x'), \forall x' \in \mathbb{R}^n, \end{aligned}$$

then h is a valid RCBF for (3).

In other words, given the set D , RCBFs ensure safety of all trajectories of the disturbed system (3). We refer the reader to [21] for the proof and note that, intuitively, this property is achieved by guaranteeing positivity of h under the worst-case disturbance, which must occur at one of the vertices defining the convex hull. Moreover, the result in Theorem 1 can be straightforwardly extended to the case where D is a finite union of convex hulls as discussed in [21], which can be leveraged to capture non-convex disturbances. As such, we can learn an estimate of the unknown disturbance d from (1) and leverage RCBFs to ensure safety with respect to that estimate.

B. Soft Actor-Critic (SAC)

The underlying RL problem considered is a policy search in an MDP defined by a tuple $(\mathcal{X}, \mathcal{U}, f, g, d, r)$, where $\mathcal{X} \subset \mathbb{R}^n$ denotes the state space, $\mathcal{U} \subset \mathbb{R}^m$ is the input space, f, g, d are as in (1), and $r: \mathcal{X} \times \mathcal{U} \times \mathcal{X} \rightarrow \mathbb{R}$ is the reward associated with each transition. Note that the state transitions for the MDP are obtained by discretizing the dynamics (1) as

$$x_{k+1} = \Delta t(f(x_k) + g(x_k)u(x_k) + d(x_k)) + x_k, \quad (4)$$

where x_k denotes the state at time step k and Δt is the time step size. We note that this choice of discretization has no effect on safety, which is ensured using RCBFs as discussed in Section III-B.

Although the proposed approach can be straightforwardly extended to any off-the-shelf policy-gradient algorithm, in this letter, we chose to focus SAC and utilize it as the underlying RL algorithm due to its state-of-the-art sample efficiency [26]. SAC maximizes an entropy objective which is given by the following

$$\pi^* = \arg \max_{\pi} \sum_t \mathbb{E}_{(x_t, u_t^{RL}) \sim \rho_{\pi}} [r(x_t, u_t^{RL}) + \alpha_e \mathcal{H}(\pi(\cdot|x_t))],$$

where x_t and u_t^{RL} denote the state and action sampled from the policy at timestep t , respectively, ρ_{π} denotes the distribution of states and actions induced by the policy π , and the term $\mathcal{H}(\pi(\cdot|x_t))$ is the entropy term which incentivizes exploration and $\alpha_e > 0$ is a weighting parameter between the two terms in the objective.

The algorithm relies on an actor-critic approach with a fitted Q-function parametrized by θ and a fitted actor π parametrized by ϕ . As such, the critic loss is given by

$$J_Q(\theta) = \mathbb{E}_{(x_t, u_t^{RL}) \sim \mathcal{D}_R} \left[\frac{1}{2} (Q_{\theta}(x_t, u_t) - (r(x_t, u_t^{RL}) + \gamma \mathbb{E}_{x_{t+1} \sim p}[V_{\bar{\theta}}(x_{t+1})]))^2 \right], \quad (5)$$

where

$$V_{\bar{\theta}}(x_t) = \mathbb{E}_{u_t^{RL} \sim \pi_{\phi}} [Q_{\bar{\theta}}(x_t, u_t^{RL}) - \alpha_e \log \pi_{\phi}(u_t^{RL}|x_t)],$$

$\mathcal{D}_R$ is the replay buffer, $\bar{\theta}$ is the target Q-network parameters and the entropy term is given by $-\log \pi_{\phi}(u_t^{RL}|x_t)$. Finally, the policy loss is given by

$$J_{\pi}(\phi) = \mathbb{E}_{x_t \sim \mathcal{D}_R} [\mathbb{E}_{u_t^{RL} \sim \pi_{\phi}} [\alpha_e \log \pi_{\phi}(u_t^{RL}|x_t) - Q_{\theta}(x_t, u_t^{RL})]]. \quad (6)$$

III. MAIN APPROACH

In this section, we introduce a differentiable RCBF-based QP which can be embedded as a safety layer in off-the-shelf policy-gradient RL algorithms. Since we choose to focus on SAC, we refer to the framework as SAC-RCBF. The first subsection discusses how we utilize GPs to learn the disturbance $D(\cdot)$ from (3). Then, we introduce the RCBF-QP which leverages the disturbance estimates to minimally alter the RL policy's actions to ensure safety of the system as pictorially depicted in Fig. 1. In Subsection III-C, we discuss how the modified RL gradient updates, which are taken with respect to the safe action, and the generation of synthetic data using the partially-learned model improve sample efficiency during training. Finally, Subsection III-D introduces novel gradient update laws to improve the transferability of SAC-RCBF.

A. Disturbance Estimation

Although RCBFs are compatible with various data-driven methods, in this letter, we choose to focus on estimating the disturbance set $D(x)$ using GPs [32]. This learning is achieved by obtaining a dataset $\mathcal{D} = \{x^{(i)}, y^{(i)}\}_{i=1}^N$ with labels $y^{(i)}$ given by $y^{(i)} = \hat{x}^{(i)} - f(x^{(i)}) - g(x^{(i)})u^{(i)}$, where $\hat{x}^{(i)}$ is the noisy

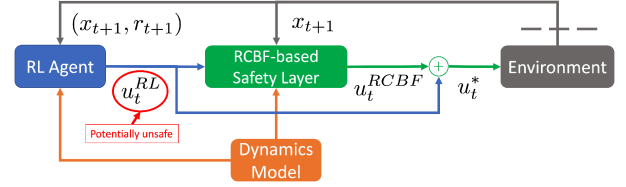


Fig. 1. A diagram of the proposed framework. At time step t the RL agent outputs the potentially unsafe control u_t^{RL} which is rendered safe using the CBF controller. The safe control input u_t^* given by (9) is then applied in the environment. Note that the dynamics model is used by both the RCBF controller to guarantee safety, and the RL agent to increase its learning's sample efficiency if possible.

measurement of the dynamics during exploration. Note that, in this case, $y^{(i)} \in \mathbb{R}^n$; therefore, we train one GP per dimension for a total of n GP models. Moreover, since we assume that the disturbance d from (1) is continuous and deterministic, we utilize the Gaussian kernel as the covariance function and optimize the hyperparameters by maximizing the log-likelihood of the collected data.

In turn, we can obtain the disturbance estimate for a query point x_* as

$$[D(x_*)]_d = \mu_d(x_*) + [-k_c \sigma_d(x_*), k_c \sigma_d(x_*)], \quad (7)$$

where $\mu_d(x_*)$ and $\sigma_d(x_*)$ are the mean and standard-deviation predictions of the d -th GP for query point x_* . The coefficient k_c is a user-chosen confidence parameter which directly affects the safety guarantees obtained. For example, $k_c = 2$ achieves a confidence of $(1 - \delta_c) = 95.5\%$ as discussed in [30]. Note that this is a different representation of $D(x)$ than the one used by Theorem 1 where the disturbance is the convex hull of a set of points. However, the desired form can be readily obtained by defining Ψ as the 2^n vectors generated by permuting the entries of D from (7).

B. RCBF-Based Safety Layer

In this subsection, we present the minimally invasive RCBF Quadratic Program (RCBF-QP) based control synthesis framework which guarantees the safety of the RL agent with dynamics as in (3). As depicted in Fig. 1, given the state, the disturbance estimate from the GPs and the action of the RL agent, this safety layer minimally alters the action so as to ensure safety. Specifically, the RCBF compensation term u^S is given by

$$\begin{aligned} (u^S(x', u^{RL}(x')), \epsilon_s) &= \arg \min_{u, \epsilon} \|u\|^2 + l \epsilon_s^2 \\ \text{s.t. } \nabla h_i(x')^\top (f(x') + g(x')(u(x') + u^{RL}(x'))) &\geq \\ &- \alpha(h_i(x')) - \min \nabla h_i(x')^\top \Psi(x') + \epsilon_s, \quad \forall i \in \mathcal{N}_s, \end{aligned} \quad (8)$$

where $u^{RL}(x') \sim \pi_{\phi}(\cdot|x')$ is the output of the RL policy at state x' , $\epsilon_s \in \mathbb{R}$ is a slack-variable that serves to ensure feasibility of the QP, $l > 0$ is a large weighting term to minimize safety violations and $\mathcal{N}_s = \{1, \dots, n_s\}$ where n_s is the number of RCBFs. As depicted in Fig. 1, the final safe action taken in the environment is given by

$$u^*(x') = u^S(x', u^{RL}(x')) + u^{RL}(x'). \quad (9)$$

Note that actuator-limit constraints can be readily added to (8). The need for a slack variable stems from cases where no control

input can satisfy the control barrier certificate and a safety violation is inevitable.

Specifically, the results stated in Lemma 1 from [10] can be extended to (8). That is, if there exists a solution with $\epsilon = 0$ for all states in $\mathcal{C}$, then safety is guaranteed with probability $(1 - \delta_c)$ at each time step. However, if no such solutions exists, we can still guarantee the safety of a larger set $\mathcal{C}_{\bar{\epsilon}} = \{x' \in \mathbb{R}^n \mid \epsilon_s(x') \leq \bar{\epsilon}\}$ with probability $(1 - \delta_c)$. We further note that other methods in the literature tackle the issue of CBF-constraint feasibility [33], [34] and that the incorporation of these methods into the proposed framework is left for future work.

As discussed in Section II-A, RCBFs are formulated with respect to the continuous-time differential inclusion (3). However, as highlighted in [35], under certain Lipschitz-continuity assumptions, solving the RCBF-based optimization program at a sufficiently high frequency ensures safety of the system during inter-triggering intervals as well. As such, if Δt from (4) is sufficiently small, we can query the RL policy and solve the RCBF-QP at the required rate for ensuring safety at all times. If Δt is relatively large, safety can still be enforced by applying a zero-order hold on the RL action and subsequently solving the RCBF-QP at the necessary rate; however, in the interest of brevity, we leave these modifications to future work.

C. Sample Efficiency

As mentioned in the Introduction, naively coupling SAC with the RCBF-QP suffers from drawbacks. Specifically, the effects of the RCBF-QP are not accounted for in the RL policy's training and the partially-learned transition model is not utilized in the training of the RL agent. Therefore, we propose two methods aimed at improving sample efficiency: (i) utilizing a differentiable safety layer which explicitly accounts for the QP's output in the RL losses via backpropagation; (ii) leveraging the partially learned dynamics to generate synthetic data that can be added to the replay buffer. Note that, although we assume knowledge of the output dynamics of the system necessary for safety, the true dynamics and the reward function may be unknown and that the safety considerations may only be local (e.g., obstacle detection). In such scenarios, the generation of synthetic data may be infeasible and only the benefits of the differentiable safety layer can be leveraged. In what follows, we discuss both of these approaches.

1) Differentiable Safety Layer: As discussed in the Introduction, the final actions taken in the environment are the altered safe actions obtained from the RCBF layer. However, the updates in equations (5) and (6) are typically taken with respect to the potentially unsafe RL actions, meaning the effects of the safety layer are included in the transition dynamics. As such, large changes in the output of the safety layer, which may occur near unsafe states and are dependent on the CBFs and the corresponding class- κ functions, can render learning difficult. Therefore, explicitly incorporating the safety layer's output in the RL losses can provide a significant learning-performance improvement, as we demonstrate in Section IV-A.

In particular, we propose utilizing the differentiable optimization framework introduced in [19], [20] to enable the propagation of gradients through the QP as in (8) using a linear system formulated from the KKT conditions. Specifically, we can backpropagate the updates in (5) and (6) through the RCBF-QP in an end-to-end fashion explicitly with respect to the safe action, improving the learning performance as discussed in Section IV.

The resulting losses for SAC are thus given by

$$J_Q(\theta) = \mathbb{E}_{(x_t, u_t^{RL}, u_t^S) \sim \mathcal{D}_R} \left[\frac{1}{2} (Q_\theta(x_t, u_t^{RL} + u_t^S) - (r(x_t, u_t^{RL} + u_t^S) + \gamma \mathbb{E}_{x_{t+1} \sim p}[V_{\bar{\theta}}(x_{t+1})]))^2 \right], \quad (10)$$

where

$$V_{\bar{\theta}}(x_t) = \mathbb{E}_{u_t^{RL} \sim \pi_\phi} [Q_{\bar{\theta}}(x_t, u_t^{RL} + u_t^S) - \alpha_e \log \pi_\phi(u_t^{RL} | x_t)],$$

and

$$J_\pi(\phi) = \mathbb{E}_{x_t \sim \mathcal{D}_R} [\mathbb{E}_{u_t^{RL} \sim \pi_\phi} [\alpha_e \log \pi_\phi(u_t^{RL} | x_t) - Q_\theta(x_t, u_t^{RL} + u_t^S)]]. \quad (11)$$

Note that the explicit dependencies of each variable have been removed for brevity. Moreover, though the RCBF increases the computation time of the losses (10) and (11), at inference time, the resulting controller only involves a forward pass of the RL policy and a single RCBF-QP solution at each time step, which is computationally inexpensive. As such, the resulting controller remains amenable to real-time control synthesis.

2) Model-Based RL: When the reward function and safety considerations are known ahead of time, we further increase the sample efficiency of SAC-RCBF by leveraging the partially learned dynamics, the reward function, and the RCBF constraints to generate short-horizon rollouts as in [36]. Compounding modelling error over long horizons motivates this usage, as data collected from longer rollouts may impede the agent from learning an optimal policy.

Specifically, at each iteration, we sample M initial states from transitions in the environment's replay buffer $\mathcal{D}_{\text{env}}$ and generate new, synthetic k -step rollouts using the dynamics prior and the disturbance estimates from the GPs. In turn, these newly generated transitions are added to another replay buffer $\mathcal{D}_{\text{model}}$ which is used to train the agent.

It is important to note that, in cases where the reward function and safety considerations are known ahead of time, one may leverage Model Predictive Control (MPC) instead of RL. However, as MPC typically requires the solution of an expensive optimization program, using RL has the advantage of being computationally efficient at test time once the policy is learned. Moreover, leveraging RL also has the additional benefit that we obtain a value function at the end of training, which can be later leveraged as a Control Lyapunov Function (CLF) or CBF itself [37] (e.g., in MPC).

D. Constraint-Agnostic Task Learning

When deploying RL in practice, certain applications may require different constraints during the training and testing stages. For example, consider the case where a differential-drive robot is tasked with navigating to a goal location while avoiding obstacles: the locations and types of the obstacles differ between the training and the test environments. Incorporating these safety considerations during training directly affects both the learned value function and the policy, which is undesirable for transferability. For example, undesired low values may be assigned around the obstacles during training. These values are due to the modification of the RL policy's action by the safety layer, which directly affects the state transition. Thus, in this section,

we present a simple yet effective approach, which we refer to as Constraint-Agnostic SAC-RCBF, for modularly learning the value function and optimal policy of the reward-driven task (i.e., independently of a subset of the constraints during training).

The effect of the safety layer can be directly observed in the Bellman update equation

$$Q(x_t, u_t^{RL} + u_t^S) = r(x_t, u_t^{RL} + u_t^S) + \gamma \mathbb{E}_{x_{t+1} \sim p} [Q(x_{t+1}, u_{t+1}^{RL} + u_{t+1}^S)]. \quad (12)$$

Specifically, note that x_{t+1} is a direct function of u_t^S , and that u_{t+1}^S affects the final action taken at x_{t+1} when considering the cost-to-go. Due to the former, entirely discarding the effects of the modifications to the action by the safety layer (i.e., setting all $u^S = 0$ in (12)) in the loss functions (10) and (11) does not yield the desired Q-function since the cost to go is still dependent on x_{t+1} through the dynamics (4).

Thus, the modification we propose is to solely discard the safety's layer output when computing the cost-to-go (i.e., only setting $u_{t+1}^S = 0$ in (12)). The resulting loss for the Q-function is given by

$$J_Q(\theta) = \mathbb{E}_{(x_t, u_t^{RL}, u_t^S) \sim \mathcal{D}_R} \left[\frac{1}{2} (Q_\theta(x_t, u_t^{RL} + u_t^S) - (r(x_t, u_t^{RL} + u_t^S) + \gamma \mathbb{E}_{x_{t+1} \sim p} [V_\theta^{mod}(x_{t+1})]))^2 \right], \quad (13)$$

where

$$V_\theta^{mod}(x_t) = \mathbb{E}_{u_t^{RL} \sim \pi_\phi} [Q_\theta(x_t, u_t^{RL}) - \alpha_e \log \pi_\phi(u_t^{RL} | x_t)],$$

and the resulting policy loss is identical to (6) as it no longer accounts for the effects of the safety layer.

Intuitively, (13) ignores the constraint considerations when computing the cost-to-go. For instance, consider the above example; this approach would allow the robot to consider the cost-to-go as if there were no obstacles and, consequently, to learn reaching the goal (i.e., the reward-driven task) in a modular fashion. It is important to note that this method does not affect the safety of the agent during training since the final action taken in the environment given by (9) remains unchanged. Moreover, we demonstrate empirically in the next section that this approach allows the RL agent to learn a value function that is independent of the safety layer, and, consequently, significantly improves performance at zero-shot transfer tasks.

IV. EXPERIMENTS

In this section, we present two sets of experiments that validate the efficacy of the proposed methods.¹ The first set of experiments aims to demonstrate the improvement in sample efficiency gained by leveraging a differentiable safety-layer as well as utilizing the partially learned model to generate synthetic data. The second set of experiments serves to validate the method proposed in Section III-D for learning the reward-driven task in a constraint-agnostic fashion.

¹The code can be found at <https://github.com/yemam3/Mod-RL-RCBF>.

A. Sample Efficiency

Here, we compare the sample efficiency of Model-Based (MB) and Model-Free (MF) SAC-RCBF against two baselines in two custom environments. We note that, while we use this terminology to distinguish between whether SAC leverages model-based rollouts, the overall framework in both cases is MB since RCBFs require partial knowledge of the dynamics. The first baseline only differs from MF SAC-RCBF in that it does not utilize a differentiable safety layer, and the second baseline leverages the compensator from [10], which is a neural-net trained to learn the behavior of the RCBF-QP during training. We did not utilize discrete-CBFs, as in [10], for the second baseline because they only apply to affine CBFs which is restrictive as discussed in the introduction. In both environments, the metric used for comparison is the agents' cumulative sum of rewards over an episode with respect to the number of episodes. Moreover, although the use of GPs only provides probabilistic guarantees, we observed that the RCBFs remain positive throughout the experiments, indicating that no safety violations occurred throughout both training and testing.

1) *Unicycle Environment*: The first environment, inspired by the Safety Gym environment [38], consists of a unicycle robot tasked with reaching a goal location while avoiding obstacles. Note that, in this section, the explicit dependence on time is dropped for brevity.

The robot is modelled using disturbed unicycle dynamics as

$$\dot{x} = \begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} \cos \theta & 0 \\ \sin \theta & 0 \\ 0 & 1 \end{bmatrix} (u + u_d),$$

where $u = [v, \omega]^\top$ is the control input with v and ω denoting the linear and angular velocities of the robot, respectively, and $u_d = -0.1 \cos(\theta) [1, 0]^\top$ is an unknown disturbance aimed at simulating a sloped surface. In turn, from the agent's perspective, the dynamics are modelled through the following differential inclusion

$$\dot{x} \in \begin{bmatrix} \cos \theta & 0 \\ \sin \theta & 0 \\ 0 & 1 \end{bmatrix} u + D(x),$$

where the disturbance set $D(x)$ is learned via GPs.

Similar to [21], we can encode a collision-avoidance constraint between the agent and a given obstacle o using the following RCBF $h_o(p(x)) = \|p(x) - p_o\|^2 - \delta^2$, where $\delta > 0$ denotes the minimum desired distance between the center of the robot $p(x)$ and the position of the obstacle p_o . Moreover, the RL observation at time step k is $s(x_k) = [x_k^\top, v_k^{g^\top}, e^{-\delta_k^g}]^\top$, where $v_k^g \in \mathbb{R}^2$ and $\delta_k^g \in \mathbb{R}_{\geq 0}$ denote the direction to the goal in the ego frame and distance to the goal, respectively. Note that this approach allows for the randomization of the goal location, since the goal is encompassed within the RL observation. However, we chose to fix the goal location in the experiment to enable a fair comparison between the various methods. We also note that the mean inference time of both SAC-RCBF approaches was approximately 0.01 seconds, which corresponds to 100 Hz and indicates the approach is indeed amenable for real-time usage as discussed in Section III-C.

Shown in Fig. 2 is a plot of the episode rewards versus training episodes for the various approaches. As shown, all approaches

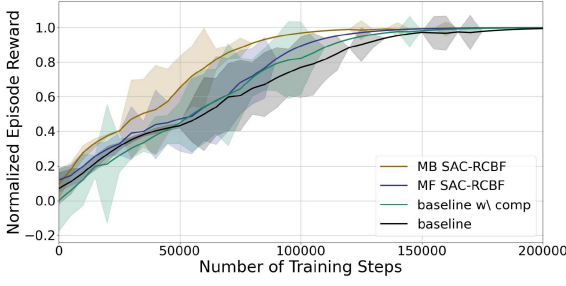


Fig. 2. Comparison of normalized episode reward versus number of training steps in the unicycle environment for the MB and MF versions of SAC-RCBF and the two baselines. Each plot is an average of ten experiments using different seeds, and the light shadowing denotes a confidence interval of two standard deviations.

converge to an optimal policy and MB SAC-RCBF performs best in terms of sample efficiency which highlights the benefit of generating synthetic data. On the other hand, the MF SAC-RCBF approach performs only slightly better than the two baselines which indicates that solely leveraging the differentiable safety layer has little benefit in this easier environment. The latter can be attributed to the fact that the behavior of the safety layer is smooth in this case as the obstacles are circular and thus the baselines, which do not explicitly account for the RCBF-QP's behavior, are able to learn the optimal policy with comparable sample efficiency.

2) *Car Following*: The second environment is based on [10] and involves a chain of five cars driving in a lane each modelled as a disturbed double integrator

$$\dot{x}_i = \begin{bmatrix} \dot{p}_i \\ \dot{v}_i \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix} x_i + \begin{bmatrix} 0 \\ 1 + d_i \end{bmatrix} u_i,$$

where p_i, v_i, u_i are the position, velocity and input acceleration of car i , respectively. In addition, $d_i = 0.1$ is a disturbance unknown to the agent that is non-zero for all cars except car 4 (i.e., $\forall i \in \{1, 2, 3, 5\}$).

The agent only controls the acceleration of the fourth car and can observe the positions, velocities and accelerations of all five cars. The leading car's behavior is aimed at simulating traffic through a sinusoidal acceleration $u_1 = 10 \sin(0.2t)$. Cars 2 and 3 behave as follows

$$u_i = \begin{cases} k_v(v_{\text{des}} - v_i) - k_b(p_{i-1} - p_i) & \text{if } |p_{i-1} - p_i| < 6 \\ k_v(v_{\text{des}} - v_i) & \text{o.w.,} \end{cases}$$

where $v_{\text{des}} = 30$ is the desired velocity and k_v and k_b are the velocity and braking gains, respectively. Lastly, the last car in the chain (i.e., car 5) has the following control input

$$u_5 = \begin{cases} k_v(v_{\text{des}} - v_5) - k_b(p_3 - p_5) & \text{if } |p_3 - p_5| < 12 \\ k_v(v_{\text{des}} - v_5) & \text{o.w.} \end{cases}$$

Moreover, two RCBFs are formulated which enforce a minimum distance between the cars three and four, and four and five, respectively $h_i(x) = \frac{1}{2}((p_i - p_4)^2 - \delta^2)$, $\forall i \in \{3, 5\}$, where $\delta = 3.5$ is the minimum distance. We note that since h_3 and h_5 have relative degree 2, a cascaded RCBF formulation as in [24] is leveraged to obtain the collision avoidance constraints. Specifically, we define $h_i^*(x) = \dot{h}_i(x) + \gamma_1 h_i(x)$, $\forall i \in \{3, 5\}$, and enforce the forward invariance of the super-zero level sets of h_i^* through RCBFs using a similar procedure to the one described

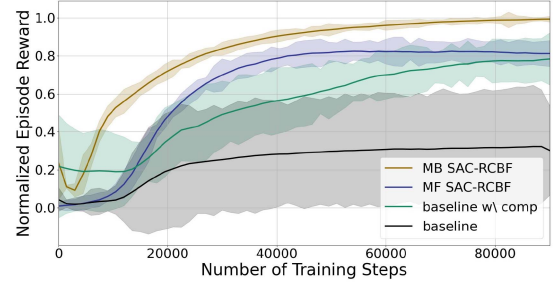


Fig. 3. Comparison of normalized episode reward versus number of training steps in the Car Following environment for the Model-Based (MB) and Model-Free (MF) versions of SAC-RCBF and the two baselines. Each plot is an average of ten experiments using different seeds, and the light shadowing denotes a confidence interval of two standard deviations.

above. The theoretical particulars of cascaded RCBFs will be discussed in future work, and we present the results without proof here. As such, the RL observation at each time step k is given by

$$s(x_k) = [x_{1k}^\top, u_{1k}, \dots, x_{4k}^\top, x_{5k}^\top, u_{5k}]^\top,$$

and the objective of the RL agent is to minimize the total control effort, which is captured through the reward function $r(u_{4k}) = -||u_{4k}||^2$, while avoiding collisions. The mean inference time of both SAC-RCBF methods was approximately 0.008 seconds (i.e., 125 Hz) for this environment.

Shown in Fig. 3 is a plot of the episode rewards versus training episodes for the different approaches. As opposed to the previous environment, both the MB and MF SAC-RCBF frameworks exhibit significantly better sample efficiency compared to the baselines. This can be attributed to the fact that the behavior of the RCBF-QP in this environment strongly affects both the dynamics and the rewards collected at each episode. Specifically, the RCBF-QP layer can cause the controlled car (Car 4) to decelerate or accelerate rapidly. Such large change in the QP's output, if not accounted for explicitly, is equivalent to a sharp transition in the dynamics which is difficult to learn. This is further highlighted by the inability of the baseline without the compensator at learning a meaningful policy in several trials as shown in Fig. 3. In turn, this signifies that explicitly accounting for the safety layer's behavior is not only critical for sample-efficient training but also for finding an optimal policy.

B. Constraint-Agnostic Task Learning

In this set of experiments, we aim to validate the Constraint-Agnostic SAC-RCBF approach proposed in Section III-D by visualizing the learned value functions for various approaches and comparing the zero-shot transfer performance in two different environments.

1) *Unicycle*: In the unicycle environment, the objective is to learn a value function for the reward-driven task (i.e., go-to-goal) that is independent of the obstacle configuration present during training. Therefore, to visually validate the efficacy of Constraint-Agnostic SAC-RCBF, shown in Fig. 4 are the learned value functions for SAC-RCBF (middle) and Constraint-Agnostic SAC-RCBF (right) trained with the obstacles denoted in red. It is clear that SAC-RCBF assigns undesirable low values around the obstacles. We note that the baselines from the previous set of experiments also assign undesirable low values around

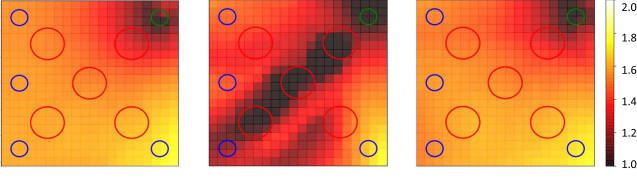


Fig. 4. Discretized plot of the learned value function for SAC trained with no obstacles and no safety layer (left), SAC-RCBF (middle), Constraint-Agnostic SAC-RCBF (right). The blue, red and green circles denote the possible starting locations for each episode, the obstacles, and the goal, respectively. The Constraint-Agnostic SAC-RCBF approach learns a value function that is almost identical to the ground-truth value function obtained by training SAC trained with no obstacles, whereas the SAC-RCBF approach differs in that it assigns low values around the obstacles.

TABLE I
ZERO-SHOT TRANSFER TO 200 DIFFERENT OBSTACLE CONFIGURATIONS IN THE UNICYCLE (TOP) AND PVTOL (BOTTOM) ENVIRONMENTS

Approach	Mean Ep. Rew.	Std Ep. Rew.	Mean Comp.
SAC trained w/o obstacles (upper bound)	7.009	1.745	0.81
Baseline	3.533	2.685	0.21
SAC-RCBF	3.223	2.413	0.17
Constraint-Agnostic SAC-RCBF	6.189	2.317	0.63

Approach	Mean Ep. Rew.	Std Ep. Rew.	Mean Comp.
SAC trained w/o obstacles (upper bound)	11.969	2.376	0.87
Baseline	7.228	4.622	0.34
SAC-RCBF	9.450	3.808	0.52
Constraint-Agnostic SAC-RCBF	10.656	3.411	0.69

the obstacles and their value function visualizations have been omitted for brevity. Since these low-value regions are dependent on the positions of the obstacles, they significantly degrade performance when transferring to environments with different obstacle configurations as we demonstrate empirically in the remainder of this section. On the other hand, Constraint-Agnostic SAC-RCBF learns a value function that is almost identical to the target making it suitable for transfer.

To further validate the approach, we tested the zero-shot transfer ability of the learned policies in 200 environments with different obstacle shapes, sizes and locations. Specifically, in each environment, four to six obstacles, where each is a triangle, square, or circle of varying size, were placed in a random configuration. Moreover, the RCBF utilized to ensure safety with respect to a given polygonal obstacle p is given by $h_p(x) = \frac{1}{2}(\min_{s \in \mathcal{S}_p} (\delta_s(x))^2 - \delta^2)$ where $\mathcal{S}_p$, $\delta_s(x)$ and δ denote the set of segments of obstacle p , the distance to segment s and the safety radius, respectively.

Shown in the top of Table I are the statistics of episode rewards and the completion percentages (i.e., if the robot reached the goal) environments for the various approaches. As expected, SAC trained with no obstacles and no safety layer solves the most environments and had a completion rate of 81% which can be treated as an upper-bound, followed by Constraint-Agnostic SAC-RCBF which had a completion average of 63%. On the other hand, both the baseline and SAC-RCBF performed quite poorly solving only 21% and 17% of the environments, respectively. These performances were mainly due to the effect that the obstacles during training had on the learned value functions and, consequently, the optimal policies.

2) *Planar Vertical Take-Off and Landing (PVTOL)*: In this experiment, we test the zero-shot transfer ability of the various approaches in an environment involving a 2D quadcopter tasked with navigating to a goal location while avoiding obstacles and staying within a certain distance of a safety pilot during training.

We utilize a modified PVTOL dynamics model in which the state is given by $x = [x_1, x_2, v_1, v_2, \theta, f]^T$, where x_i , v_i denote the position and velocity along axis i , respectively, θ is the orientation of the robot and f is the thrust. As such, the dynamics are as follows

$$\dot{x} = [v_1, v_2, -\sin(\theta)f, \cos(\theta)f - 1, \omega + d_\omega, u_f]^T.$$

Differently than the traditional PVTOL model [39], we assume direct control of the angular velocity ω and the derivative of the thrust denoted by u_f . We note that these assumptions are reasonable since a popular controller design approach for PVTOLs is to use the moment M to track a desired ω , and we model the tracking error as a disturbance on ω 's dynamics denoted by d_ω . Moreover, controlling the derivative of the thrust can be straightforwardly achieved by augmenting the system with artificial thrust dynamics as performed here.

As such, the RCBF utilized for obstacle avoidance is given by $h_o(p(x)) = \|p(x) - p_o\|^2 - \delta^2$, where $\delta > 0$ denotes the minimum desired distance between $p(x) = [x_1, x_2]^T$ and the position of the obstacle p_o . Similar to the Car Following environment, since h_o has relative degree 3, a cascaded RCBF formulation as in [24] is leveraged to obtain the collision avoidance constraints. The safety operator is assumed to have noisy single-integrator dynamics along the x_1 -axis and tracks the drone via a low-gain proportional controller. The RCBF enforcing the 2D quadcopter to keep a certain distance from the operator is given by $h_h(x_1) = \delta^2 - (x_1 - x_h)^2$. Moreover, similarly to the unicycle environment, the RL observation at time step k is given by $s(x_k) = [x_k^T, v_k^{gT}, e^{-\delta_k^g}]^T$, where $v_k^g \in \mathbb{R}^2$ and $\delta_k^g \in \mathbb{R}_{\geq 0}$ denote the direction to the goal in the quadcopter's frame and distance to the goal, respectively. We note that, in this environment, the mean inference time of both SAC-RCBF approaches was approximately 0.015 seconds which corresponds to 66 Hz.

Shown in the bottom of Table I are the zero-shot transfer results of the policies learned by the different methods to 200 randomized environments with different numbers and locations of obstacles as well as the presence of a safety operator constraint. Similar to the Unicycle environment, we obtain a performance upper-bound by training SAC in an environment with no obstacles nor a safety operator and thus without a safety layer. As expected, this approach performed best at zero-shot transfer with a completion rate of 87%. Moreover, Constraint-Agnostic SAC-RCBF yields a 33% improvement over SAC-RCBF by completing 69% of the environments as compared to 52%. As such, this experiment validates that Constraint-Agnostic SAC-RCBF is suitable for transfer tasks.

V. CONCLUSION

In this letter, we introduce a novel sample-efficient framework that combines RL with a RCBF-based layer to enforce safety during training. The resulting SAC-RCBF approach is empirically validated in simulated environments and is shown to improve both the sample efficiency and steady-state performance during RL training. Moreover, we propose an approach for modularly learning the reward-driven task independently of a subset of the safety constraints resulting in learned policies that can be leveraged for transfer to different environments involving different safety constraints. A relevant direction of future work is to utilize the proposed framework for the reinforcement learning of non-safety critical CBFs.

REFERENCES

- [1] J. Kober, J. A. Bagnell, and J. Peters, "Reinforcement learning in robotics: A survey," *Int. J. Robot. Res.*, vol. 32, no. 11, pp. 1238–1274, 2013.
- [2] K. Arulkumaran, M. P. Deisenroth, M. Brundage, and A. A. Bharath, "Deep reinforcement learning: A brief survey," *IEEE Signal Process. Mag.*, vol. 34, no. 6, pp. 26–38, Nov. 2017.
- [3] B. Thananjeyan et al., "Recovery RL: Safe reinforcement learning with learned recovery zones," *IEEE Robot. Automat. Lett.*, vol. 6, no. 3, pp. 4915–4922, Jul. 2021.
- [4] A. I. Cowen-Rivers et al., "Samba: Safe model-based & active reinforcement learning," vol. 111, pp. 179–203, 2022.
- [5] L. Brunke et al., "Safe learning in robotics: From learning-based control to safe reinforcement learning," *Annu. Rev. Control, Robot., Auton. Syst.*, vol. 5, pp. 411–444, 2022.
- [6] E. Altman, "Constrained markov decision processes with total cost criteria: Lagrangian approach and dual linear program," *Math. Methods Operations Res.*, vol. 48, no. 3, pp. 387–417, 1998.
- [7] P. Geibel and F. Wysotzki, "Risk-sensitive reinforcement learning applied to control under constraints," *J. Artif. Intell. Res.*, vol. 24, pp. 81–108, 2005.
- [8] Y. Chow, M. Ghavamzadeh, L. Janson, and M. Pavone, "Risk-constrained reinforcement learning with percentile risk criteria," *J. Mach. Learn. Res.*, vol. 18, no. 1, pp. 6070–6120, 2017.
- [9] Z. Marvi and B. Kiumarsi, "Safe off-policy reinforcement learning using barrier functions," in *Proc. IEEE Amer. Control Conf.*, 2020, pp. 2176–2181.
- [10] R. Cheng, G. Orosz, R. M. Murray, and J. W. Burdick, "End-to-end safe reinforcement learning through barrier functions for safety-critical continuous control tasks," in *Proc. AAAI Conf. Artif. Intell.*, 2019, vol. 33, pp. 3387–3395.
- [11] F. Berkenkamp, M. Turchetta, A. P. Schoellig, and A. Krause, "Safe model-based reinforcement learning with stability guarantees," in *Proc. 31st Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 908–919.
- [12] Y. Chow et al., "A lyapunov-based approach to safe reinforcement learning," in *Proc. 32nd Int. Conf. Neural Inf. Process. Syst.*, 2018, pp. 8103–8112.
- [13] J. Choi, F. Castaneda, C. J. Tomlin, and K. Sreenath, "Reinforcement learning for safety-critical control under model uncertainty, using control lyapunov functions and control barrier functions," in *Proc. Robot.: Sci. Syst. XVI*, M. Toussaint, A. Bicchi, and T. Hermans, Eds., 2020.
- [14] Y. Chow, O. Nachum, A. Faust, E. Duenez-Guzman, and M. Ghavamzadeh, "Lyapunov-based safe policy optimization for continuous control," 2019, *arXiv:1901.10031*.
- [15] E. Altman, *Constrained Markov Decision Processes*, vol. 7. Boca Raton, FL, USA: CRC, 1999.
- [16] J. F. Fisac, A. K. Akametalu, M. N. Zeilinger, S. Kaynama, J. Gillula, and C. J. Tomlin, "A general safety framework for learning-based control in uncertain robotic systems," *IEEE Trans. Autom. Control*, vol. 64, no. 7, pp. 2737–2752, Jul. 2019.
- [17] Z. Li, U. Kalabić, and T. Chu, "Safe reinforcement learning: Learning with supervision using a constraint-admissible set," in *Proc. IEEE Annu. Amer. Control Conf.*, 2018, pp. 6390–6395.
- [18] J. Zeng, B. Zhang, and K. Sreenath, "Safety-critical model predictive control with discrete-time control barrier function," in *Proc. IEEE Amer. Control Conf.*, 2021, pp. 3882–3889.
- [19] B. Amos and J. Z. Kolter, "OptNet: Differentiable optimization as a layer in neural networks," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 136–145.
- [20] A. Agrawal et al., "Differentiable convex optimization layers," *Adv. Neural Inf. Process. Syst.*, 2019, pp. 9562–9574.
- [21] Y. Emam, P. Glotfelter, and M. Egerstedt, "Robust barrier functions for a fully autonomous, remotely accessible swarm-robotics testbed," in *Proc. IEEE 58th Conf. Decis. Control*, 2019, pp. 3984–3990.
- [22] Y. Li and C. Liu, "Applications of multirotor drone technologies in construction management," *Int. J. Construction Manage.*, vol. 19, no. 5, pp. 401–412, 2019.
- [23] S. J. Kim, Y. Jeong, S. Park, K. Ryu, and G. Oh, "A survey of drone use for entertainment and AVR (augmented and virtual reality)," in *Augmented Reality and Virtual Reality*. Berlin, Germany: Springer, 2018, pp. 339–352.
- [24] G. Notomista and M. Egerstedt, "Persistification of robotic tasks," *IEEE Trans. Control Syst. Technol.*, vol. 29, no. 2, pp. 756–767, Mar. 2021.
- [25] V. Makoviychuk et al., "ISAAC GYM: High performance GPU based physics simulation for robot learning," in *Proc. 35th Conf. Neural Inf. Process. Syst. Datasets Benchmarks Track*, 2021.
- [26] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 1861–1870.
- [27] A. D. Ames, J. W. Grizzle, and P. Tabuada, "Control barrier function based quadratic programs with application to adaptive cruise control," in *Proc. IEEE 53rd Conf. Decis. Control*, 2014, pp. 6271–6278.
- [28] X. Xu, P. Tabuada, J. W. Grizzle, and A. D. Ames, "Robustness of control barrier functions for safety critical control," *IFAC-PapersOnLine*, vol. 48, no. 27, pp. 54–61, 2015.
- [29] A. D. Ames, X. Xu, J. W. Grizzle, and P. Tabuada, "Control barrier function based quadratic programs with application to automotive safety systems," in *Proc. Amer. Control Conf.*, 2015, pp. 4542–4548.
- [30] Y. Emam, P. Glotfelter, S. Wilson, G. Notomista, and M. Egerstedt, "Data-driven robust barrier functions for safe, long-term operation," *IEEE Trans. Robot.*, vol. 38, no. 3, pp. 1671–1685, Jun. 2022.
- [31] J. Cortes, "Discontinuous dynamical systems," *IEEE Control Syst. Mag.*, vol. 28, no. 3, pp. 36–73, Jun. 2008.
- [32] C. E. Rasmussen, "Gaussian processes in machine learning," in *Summer School on Machine Learning*. Berlin, Germany: Springer, 2003, pp. 63–71.
- [33] J. Zeng, B. Zhang, Z. Li, and K. Sreenath, "Safety-critical control using optimal-decay control barrier function with guaranteed point-wise feasibility," in *Proc. Amer. Control Conf.*, 2021, pp. 3856–3863.
- [34] E. Squires, P. Pierpaoli, and M. Egerstedt, "Constructive barrier certificates with applications to fixed-wing aircraft collision avoidance," in *Proc. IEEE Conf. Control Technol. Appl.*, 2018, pp. 1656–1661.
- [35] V. Dhiman, M. J. Khojasteh, M. Franceschetti, and N. Atanasov, "Control barriers in bayesian learning of system dynamics," *IEEE Trans. Autom. Control*, vol. 68, no. 1, pp. 214–229, 2021.
- [36] M. Janner, J. Fu, M. Zhang, and S. Levine, "When to trust your model: Model-based policy optimization," *Adv. Neural Inf. Process. Syst.*, vol. 32, pp. 12519–12530, 2019.
- [37] M. Ohnishi, G. Notomista, M. Sugiyama, and M. Egerstedt, "Constraint learning for control tasks with limited duration barrier functions," *Automatica*, vol. 127, 2021, Art. no. 109504.
- [38] A. Ray, J. Achiam, and D. Amodei, "Benchmarking safe exploration in deep reinforcement learning," 2019. [Online]. Available: <https://cdn.openai.com/safexp-short.pdf>
- [39] A. Zavala-Rio, I. Fantoni, and R. Lozano, "Global stabilization of a pvtol aircraft model with bounded inputs," *Int. J. Control*, vol. 76, no. 18, pp. 1833–1844, 2003.